NΛCO — Arquitetura de Software e Especificação Técnica

Documento mestre do sistema • MVP

Documentação técnica do projeto NΛCO

09 de novembro de 2026

Sumário

Controle do documento **4**
Convenções de leitura 5

1. Resumo executivo **5**

2. Objetivos, princípios e critérios de sucesso **6**
2.1 Objetivos do produto 6
2.2 Objetivos técnicos 7
2.3 Princípios arquiteturais 7

3. Visão do produto e limites do MVP **8**
3.1 Atores 8
3.2 Capacidades incluídas 8
3.3 Fora de escopo e não presumir 9

4. Requisitos funcionais e regras de negócio **9**
4.1 Requisitos funcionais 9
4.2 Regras de negócio 11
4.3 Matriz de cancelamento aprovada 11

5. Fluxos de negócio ponta a ponta **12**
5.1 Cadastro, login e sessão 12
5.2 Solicitação de RESPONSAVEL 12
5.3 Restaurante e horário 13
5.4 Catálogo e estoque 13
5.5 Carrinho, checkout e pedido 13
5.6 Confirmação, recusa, preparo e conclusão 14
5.7 Visualização e inbox de pedidos 14
5.8 Avaliação de produto 15

5.9 Administração 15

6. Arquitetura de solução **15**
6.1 Estilo arquitetural 15
6.2 Diagrama de contexto 16
6.3 Diagrama de contêineres e responsabilidades 16
6.4 Camadas internas 17
6.5 Módulos de domínio 17
6.6 Estrutura de repositório recomendada 19

7. Stack tecnológica e justificativas **19**
7.1 Redis: responsabilidade e limites 21
7.2 Armazenamento de imagens 21

8. Modelo de domínio e persistência **22**
8.1 Entidades centrais 22
8.2 Relacionamentos 25
8.3 Restrições e índices obrigatórios/conceituais 25
8.4 Dinheiro e precisão 26
8.5 Horários e timezone 26
8.6 Normalização de nome 27

9. Arquitetura de pedidos, estoque e idempotência **27**
9.1 Estados canônicos 27
9.2 Máquina de estados e autorização 27
9.3 Estoque 27
9.4 Idempotência de criação 28
9.5 Compatibilização QU-60 — resolução consolidada 28

10. Contratos de API **29**
10.1 Padrões gerais 29
10.2 Inventário de rotas orientador 29
10.3 Exemplo de criação de pedido 31
10.4 Envelope de erro 32

11. Segurança e privacidade **33**
11.1 Senhas e tokens 33
11.2 Armazenamento no cliente 34
11.3 Autorização 34
11.4 Rate limiting e defesa 34
11.5 Dados que não podem ir aos logs 35
11.6 Auditoria e retenção 35

12. Frontend mobile e web **35**
12.1 Mobile — cliente 35
12.2 Web — responsável/admin 36

13. Segurança operacional e integração **36**
13.1 Política de retries (RNF-02 operacionalizado) 36
13.2 Disponibilidade, backup e recuperação 36
13.3 Observabilidade 37

14. Estratégia de testes e qualidade **37**
14.1 Pirâmide de testes 37
14.2 Fluxos E2E obrigatórios 38
14.3 Testes críticos de segurança e concorrência 38
14.4 Critérios de qualidade e gate de release 39

15. RNF-01 — Desempenho e integridade sob carga **39**
15.1 Operações oficiais 39
15.2 Limites oficiais 39
15.3 Carga sustentada 40
15.4 Matriz quantitativa de homologação 40
15.5 Evidências obrigatórias 40

16. CI/CD, ambientes e release **40**
16.1 Ambientes 40
16.2 Pipeline sugerido 41
16.3 Migrations 41

17. Gestão de riscos técnicos **41**

18. Itens remanescentes para contrato, implementação e operação **43**
18.1. Contratos que ainda precisam de detalhamento antes do freeze da OpenAPI 43
18.2. Materialização de schema e implementação — trabalho futuro esperado 44
18.3. Validações operacionais antes de staging/produção . . 44
18.4. Decisões fechadas que não devem voltar à lista de pendências 45

19. Estratégia de implementação por fatias verticais **46**

20. Definição de pronto (DoD) e gates **46**
20.1 Marcos de liberação 47

21. Glossário **47**

22. Fontes e governança documental **48**

Apêndices de rastreabilidade integral **49**
Apêndice A — Critérios de aceitação, cenários e rastreabilidade preservados 49

Critérios 1. RF-01 — Cadastro e autenticação 49
Critérios 2. RF-02 — Avaliação de produto 49
Critérios 3. RF-03 — Pedido demonstrativo 49
Critérios 4. RF-04 — Painel 50
Critérios 5. RF-05/RF-09 — Catálogo 50
Critérios 6. RF-06/RF-07 — Papéis e restaurantes 50
Critérios 7. RNF-01 51
Critérios 8. RNF-02 51
8. Cenários de teste relacionados 51
Cenários 1. Conta, papéis e autorização 51
Cenários 2. Restaurante, catálogo e painel 51
Cenários 3. Pedido e avaliação 52
Cenários 4. Idempotência e falhas 53
Cenários 5. RNF-01: carga, concorrência e instrumentação 53
Cenários 6. RNF-02 53
9. Dependências entre requisitos 53
10. Matriz de rastreabilidade resumida 55
11. Itens ainda a materializar (estado consolidado) 58
Apêndice B — Registro integral das decisões QU-01 a QU-73 59
QU-01 a QU-19 — Critérios centrais de desempenho . . 59
QU-20 a QU-28 — RPS e concorrência de estoque 61
QU-29 a QU-39 — Idempotência, atomicidade e falhas . 62
QU-40 a QU-50 — Preparação, observabilidade e overhead 64
QU-51 a QU-56 — Fluxo de consulta e escopo da chave 66
QU-57 a QU-62 — Payload e persistência idempotente . 66
QU-63 a QU-67 — Concorrência entre ações de pedido . 67
QU-68 a QU-70 — Recuperação 68
QU-71 a QU-73 — Relatório e encerramento 68
Resumo quantitativo 69

Status documental: versão consolidada para orientar o início da implementação. Requisitos e decisões complementares já aprovados foram propagados para este documento; os contratos definidos orientarão a futura OpenAPI. Não é evidência de implementação, homologação ou prontidão para produção.

Controle do documento

Campo	Valor
Sistema	NΛCO (pronúncia: NACO)
Documento	Arquitetura de software e especificação técnica mestre
Versão	1.2

Campo	Valor
Data-base	09/11/2026
Escopo	MVP mobile + web + API + persistência + operações
Arquitetura	Monólito modular
Repositório conhecido	`Gabrielhanel/naco`
Fontes	Especificação consolidada, modelo de dados/API, registro QU-01–QU-73, auditoria documental, adendo de decisões complementares e decisões Q63–Q103 de imagens
Estado do software	Não declarado como implementado; este documento é planejamento/especificação

Convenções de leitura

- **Aprovado:** requisito ou decisão expressamente registrada nas fontes do projeto.
- **Padrão técnico adotado:** escolhas de implementação fechadas no adendo de 02/11/2026 e nas decisões complementares Q63–Q103 de imagens, consolidadas em documento próprio.
- **Recomendação de arquitetura:** orientação técnica para concretizar uma decisão, sem alterar regra de negócio.
- **Pendência de contrato:** ponto que deve ser resolvido antes de publicar/finalizar a OpenAPI ou implementar a operação afetada.
- **Fora do MVP:** capacidade explicitamente excluída do primeiro corte.

A documentação-base e as regras de domínio aprovadas têm precedência. Este documento organiza e aprofunda as decisões, mas não autoriza alterar requisitos preexistentes por conveniência técnica. Quando houver conflito, a implementação da operação afetada deve aguardar a resolução formal.

1. Resumo executivo

O NΛCO é uma plataforma demonstrativa de descoberta de restaurantes, consulta de cardápios, simulação de pedidos e gestão de estabelecimentos. O produto tem duas experiências principais: aplicativo mobile para clientes e aplicação web para responsáveis por restaurantes. Ambos consomem uma API REST única, responsável por au-

tenticação, autorização, regras de domínio, integridade transacional e persistência.

O MVP será construído como **monólito modular**, não como conjunto de microsserviços. A separação interna será por módulos de domínio, com camadas explícitas e dependências controladas. PostgreSQL é a fonte de verdade; Redis atende cache de consultas públicas e limitação de requisições, sem substituir consistência do banco. A comunicação de atualização de pedidos será polling HTTP a cada 15 segundos.

O objetivo é entregar uma aplicação funcional e, ao mesmo tempo, um laboratório de engenharia: arquitetura evolutiva, contratos explícitos, persistência consistente, autenticação segura, testes automatizados, concorrência, observabilidade, recuperação, CI/CD e campanha de desempenho verificável.

O que o MVP entrega: contas e papéis; cadastro e gestão de restaurantes; horários e publicação; categorias e catálogo; preços e estoque; carrinho local; checkout e ciclo de vida de pedidos simulados; histórico; avaliação de produtos comprados; painel de indicadores; administração controlada; API documentada; testes e operação básica.

O que o MVP não entrega: pagamento real, entregadores reais, rastreamento GPS, prazo real de entrega, emissão fiscal, cupons, fidelidade, recomendações automatizadas, MFA, carrinho sincronizado entre dispositivos, WebSocket/SSE, filas, microsserviços ou exceções de calendário/feriados.

2. Objetivos, princípios e critérios de sucesso

2.1 Objetivos do produto

1. Permitir ao cliente descobrir estabelecimentos publicados, consultar produtos disponíveis e simular uma compra.
2. Permitir ao responsável operar um ou vários restaurantes, mantendo isolamento entre estabelecimentos.
3. Preservar a história comercial demonstrativa: snapshots de nome, preço, cliente e endereço no momento do pedido.
4. Demonstrar integridade em fluxos críticos: estoque, confirmação, cancelamento, idempotência e autorização.
5. Oferecer experiência mobile para cliente e web para gestão, apoiadas pelo mesmo backend.

2.2 Objetivos técnicos

- Separação por domínio e baixo acoplamento dentro de um monólito.
- Contratos REST explícitos, versionáveis e descritos em OpenAPI.
- Validação no cliente para UX e no servidor para segurança e integridade.
- Fonte de verdade transacional no PostgreSQL.
- Autorização por usuário, papel, propriedade e escopo do restaurante verificada no servidor.
- Testes reais de integração para constraints, transações e concorrência.
- CI reproduzível e ambiente local por Docker Compose.
- RNF-01 testável com campanha, evidências e critérios quantitativos.

2.3 Princípios arquiteturais

Princípio	Aplicação no NΛCO
Simplicidade evolutiva	Monólito modular primeiro; microsserviços somente com evidência de necessidade.
Domínio antes da tecnologia	Regras de pedido/estoque não residem em componentes de UI nem em controllers.
Fonte de verdade única	PostgreSQL é autoridade para usuários, catálogo, pedidos, estoque e permissões.
Fail closed	Falhas de autenticação/autorização e, quando necessário, de rate limiting não devem abrir acesso.
Imutabilidade histórica	Alterações no catálogo/perfil não reescrevem snapshots de pedidos concluídos ou em andamento.
Consistência transacional	Mudanças de estoque, estado e eventos relacionados são atômicas.
Contrato explícito	DTOs públicos não expõem modelos ORM diretamente.

Princípio	Aplicação no NΛCO
Observabilidade segura	Logs e métricas ajudam a diagnosticar sem armazenar segredos ou dados sensíveis desnecessários.
Testabilidade	Regras críticas são verificáveis sem interface e integradas ao banco real.

3. Visão do produto e limites do MVP

3.1 Atores

Ator	Responsabilidade
Visitante	Descobrir restaurantes e cardápios públicos; iniciar cadastro/login.
Cliente (`CLIENTE`)	Gerenciar nome próprio, manter carrinho local, criar pedidos, consultar histórico e avaliar itens de pedidos concluídos.
Responsável (`RESPONSAVEL`)	Administrar restaurantes aos quais está associado, catálogo, disponibilidade, pedidos recebidos e indicadores. Pode gerenciar vários restaurantes.
Administrador (`ADMIN`)	Administrar solicitações e operações privilegiadas conforme matriz; uma única conta, criada manualmente, exclusiva e auditada.
Operador/desenvolvedor	Preparar ambiente, seed administrativo controlado, infraestrutura, backup, observabilidade e liberação. Não é um papel público adicional.

3.2 Capacidades incluídas

- Cadastro público atribui automaticamente `CLIENTE`.
- Solicitação e aprovação administrativa para concessão de `RESPONSAVEL`.

- Conta `ADMIN` única e exclusiva, sem cadastro público.
- Cadastro/edição/publicação de restaurante, com um único responsável por restaurante.
- Horários semanais e timezone do restaurante.
- Categorias e ofertas específicas por restaurante.
- Estoque e preço por oferta.
- Carrinho local mobile com um restaurante por vez.
- Criação idempotente de pedido simulado.
- Confirmação, recusa, preparo, saída para entrega demonstrativa, conclusão e cancelamento conforme matriz.
- Histórico e visualização individual de pedidos recebidos.
- Avaliações `PRODUCT_ONLY`, uma por item comprado, somente após conclusão.
- Dashboard com produtos vendidos e avaliações recebidas; sem faturamento.
- Rate limiting, segurança de sessão, logs, métricas, backup e testes.

3.3 Fora de escopo e não presumir

Pagamento, gateway, dinheiro real, liquidação, split, operação real de entregador, roteirização, GPS, previsão de entrega, emissão fiscal, cupom, fidelidade, promoções complexas, recomendação automatizada, MFA, multi-endereços salvos, carrinho persistido no servidor, sincronização de carrinho entre dispositivos, feriados/exceções de horário, push notifications, WebSocket/SSE, mensageria e microservices.

4. Requisitos funcionais e regras de negócio

Os IDs RF-01 a RF-09 e RN-01 a RN-09 são preservados. A tabela resume a intenção; os critérios detalhados e cenários de teste devem continuar rastreados nos documentos de requisitos.

4.1 Requisitos funcionais

ID	Capacidade	Comportamento essencial
RF-01	Conta e autenticação	Cadastro/login; e-mail global único; CLIENTE automático; sessão com refresh/revogação.
RF-02	Avaliação	Avaliar somente produto adquirido; vínculo com OrderItem próprio; pedido CONCLUIDO; nota 1–5; comentário opcional; uma por item.
RF-03	Pedido/entrega simulados	Pedido de um restaurante; inicia CRIADO; transições autorizadas; sem pagamento/entrega real.
RF-04	Dashboard	Responsável consulta indicadores apenas do próprio restaurante.
RF-05	Gestão do catálogo	Criar, editar, desativar/excluir oferta respeitando histórico e autorização.
RF-06	Papéis	CLIENTE automático; RESPONSAVEL após solicita-ção/aprovação; ADMIN exclusivo e manual.
RF-07	Restaurante	Criar/editar/publicar/despublicar; vários restaurantes por responsável; um responsável por restaurante.

ID	Capacidade	Comportamento essencial
RF-08	Preço e estoque	Preço positivo; estoque não negativo; baixa na confirmação; restauração única no cancelamento elegível.
RF-09	Alteração de produto	Alterar nome, preço, imagem, descrição e categoria sem alterar snapshots históricos.

4.2 Regras de negócio

- **RN-01:** pedido/carrinho contém produtos de um único restaurante.
- **RN-02:** não pode haver produto com nome duplicado dentro do mesmo restaurante. A forma de comparação aprovada pelo adendo usa `normalized_name` derivado no servidor.
- **RN-03:** responsável só gerencia restaurantes/recursos que lhe pertencem ou para os quais possui autorização ativa.
- **RN-04:** conta altera somente seus dados pessoais, salvo ação administrativa autorizada.
- **RN-05:** avaliação `PRODUCT_ONLY`, do próprio cliente, item de pedido concluído, uma por item, nota 1–5.
- **RN-06:** produto sem estoque não pode ser incluído no carrinho; continua visível como indisponível. Estoque só baixa na confirmação.
- **RN-07:** oferta removida não pode entrar em novos pedidos; referências históricas permanecem válidas.
- **RN-08:** adicionar ao carrinho exige autenticação e restaurante aberto; criar pedido exige restaurante aberto e publicado.
- **RN-09:** remover/alterar item atualiza subtotal e total localmente.

4.3 Matriz de cancelamento aprovada

Estado atual	Cliente	Responsável
`CRIADO`	Pode cancelar	Não; pode recusar

Estado atual	Cliente	Responsável
CONFIRMADO_RESTAURANTE	Pode cancelar	Pode cancelar
EM_PREPARO	Pode cancelar	Não
SAIU_PARA_ENTREGA	Não	Não
CONCLUIDO	Não	Não
RECUSADO	Não	Não
CANCELADO	Não	Não

Se cancelamento ocorrer após confirmação, estado, evento e reposição do estoque são persistidos atomicamente e apenas uma vez.

5. Fluxos de negócio ponta a ponta

5.1 Cadastro, login e sessão

Uma conta pode ter CLIENTE e RESPONSAVEL ao mesmo tempo. No MVP não há seletor de identidade/papel no login: mobile atende fluxos CLIENTE e web atende gestão RESPONSAVEL; cada endpoint valida papel e ownership no backend.

1. Cliente envia nome, e-mail e senha ao cadastro público.
2. API valida sintaxe, normaliza e-mail conforme política única, verifica unicidade e força da senha.
3. Senha é derivada com Argon2id; nunca é persistida em texto puro.
4. Usuário e papel CLIENTE são criados no mesmo fluxo transacional.
5. Login verifica hash e limita tentativas por conta/IP.
6. API cria sessão, access token curto e refresh token opaco.
7. Refresh é rotativo; reutilização de token anterior invalida a família de sessão.
8. Logout revoga sessão/refresh no servidor; clientes limpam estado local.
9. Todas as operações protegidas verificam sessão ativa e autorização atual no servidor.

Regra de implementação: JWT não deve ser fonte definitiva de papéis ou ownership. Deve carregar apenas identificadores e claims técnicas mínimas; permissão real é consultada no backend.

5.2 Solicitação de RESPONSAVEL

1. Usuário autenticado com CLIENTE solicita papel.

2. Sistema verifica cooldown de 7 dias após rejeição e não permite duplicidade de solicitação pendente.
3. Solicitação é persistida com estado e timestamps.
4. ADMIN consulta solicitações pendentes.
5. Aprovação e concessão do papel acontecem na mesma transação; rejeição grava motivo opcional e data.
6. Todas as decisões/grants/removals geram evento de auditoria.
7. Remover RESPONSAVEL exige transferir todos os restaurantes para outro responsável existente antes da remoção; não excluir restaurante automaticamente.

5.3 Restaurante e horário

1. RESPONSAVEL cadastra restaurante; API define `responsible_user_id` a partir da sessão.
2. Novo restaurante começa `UNPUBLISHED`.
3. Dados incluem timezone IANA e grade semanal de horários.
4. Publicação é operação separada do funcionamento. Restaurante publicado e fechado pode ser listado, mas não aceita pedido.
5. Despublicação é bloqueada com pedidos ativos em CRIADO, CONFIRMADO_RESTAURANTE, EM_PREPARO ou SAIU_PARA_ENTREGA.
6. Horário ausente/inválido implica fechado; horários são avaliados no timezone do restaurante.

5.4 Catálogo e estoque

1. O responsável cria uma oferta (`RestaurantProduct`) vinculada a `ProductBase` e categoria do mesmo restaurante.
2. `display_name` começa com `ProductBase.name`; `normalized_name` é produzido no servidor.
3. A unicidade de nome é aplicada por restaurante inclusive para oferta desativada.
4. Preço é BRL, positivo e armazenado como DECIMAL(12,2); estoque é inteiro e não negativo.
5. Estoque zero não remove o item do catálogo, mas bloqueia compra.
6. Alterações no catálogo não mudam snapshot de `OrderItem` já gravado.
7. Oferta referenciada por pedido deve ser desativada ou ter exclusão física restringida.

5.5 Carrinho, checkout e pedido

1. Mobile mantém um carrinho local por usuário e por restaurante.
2. Cliente só adiciona produto disponível e de restaurante aberto.

3. Checkout envia restaurante, endereço e itens com quantidades e `expectedUnitPrice`.
4. Cliente gera uma UUID v4 `Idempotency-Key` por intenção lógica.
5. Backend verifica sessão, titularidade do carrinho, publicação/horário, disponibilidade e preços atuais.
6. Divergência de preço produz 409 `PRICE_CHANGED`; o cliente pede revisão e nova intenção deve usar chave nova.
7. Registro idempotente, Order e OrderItems são criados na mesma transação. Nenhum estoque é baixado ou reservado nesta fase.
8. Endereço, nome e e-mail do cliente são congelados no pedido.
9. Timeout no cliente leva a consultar a chave antes de qualquer nova tentativa.

5.6 Confirmação, recusa, preparo e conclusão

- Responsável recebe pedidos por restaurante ou inbox consolidada.
- Lista é resumida e ordenada por ativos primeiro, mais recentes primeiro, ID descendente como desempate.
- Detalhes mostram itens, snapshots, cliente, endereço e eventos cronológicos.
- `PATCH /api/orders/{orderId}/status` é o único contrato de transição.
- Confirmação bloqueia ofertas em ordem determinística, revalida estoque e, em uma transação, baixa todos os itens, muda status e cria evento.
- Falha de estoque reverte integralmente e pedido continua CRIADO; erro 409 `INSUFFICIENT_STOCK`.
- Recusa em CRIADO não baixa estoque.
- Avanço manual segue sequência validada; update condicional previne concorrência.
- Cancelamento após confirmação muda status, cria evento e restaura estoque na mesma transação.

5.7 Visualização e inbox de pedidos

- `GET /api/restaurants/{restaurantId}/orders`: resumo para responsável do restaurante.
- `GET /api/me/orders/inbox`: pedidos de todos os restaurantes geridos, com contadores por restaurante e global.
- `GET /api/orders/{orderId}`: detalhe do pedido, sem mutação.
- `POST /api/orders/{orderId}/views`: registra a primeira visualização por responsável; idempotente, preserva `first_viewed_at`.
- Polling de 15 segundos, pausado quando tela não está visível; atualizar ao voltar.

- Cada responsável possui estado próprio de leitura (`OrderView`), não compartilhado com outros responsáveis.
- Pedidos encerrados saem do contador de não lidos, mas histórico de visualização permanece.

A listagem e os detalhes são leituras puras. A visualização é registrada separadamente por `POST /api/orders/{orderId}/views`, evitando efeito colateral em GET.

5.8 Avaliação de produto

- Endpoint adotado no adendo: `POST /api/order-items/{orderItemId}/reviews`.
- Cliente dono do pedido, após `CONCLUIDO`, cria uma avaliação por OrderItem.
- Nota inteira 1–5; comentário opcional.
- Não implementar review de restaurante, edição ou exclusão no MVP.
- O caminho legado `/api/restaurants/{id}/reviews` deve ser marcado como substituído no contrato consolidado.

5.9 Administração

- Uma conta ADMIN, exclusiva, sem cadastro público; seed/script protegido.
- Aprovar/rejeitar pedidos de papel, transferir responsável, remover RESPONSAVEL após transferências e auditar ações.
- Ações sensíveis que dependam de autorização do proprietário do projeto não devem gerar um segundo fluxo de aprovação dentro do produto; o procedimento externo precisa ser definido operacionalmente.
- Permissões e ownership são verificados em todas as requisições, mesmo quando o frontend esconde botões.

6. Arquitetura de solução

6.1 Estilo arquitetural

Monólito modular orientado a domínio, com uma API Node/Express e módulos separados por contexto. Um único processo de aplicação pode ser escalado horizontalmente sem decompor os domínios em serviços independentes. A separação interna cria fronteiras para uma eventual extração futura, mas não implica implantação distribuída no MVP.

6.2 Diagrama de contexto

```
flowchart LR
  C[Cliente] --> M[Aplicativo mobile]
  R[Responsável] --> W[Web de gestão]
  A[Administrador] --> W
  M --> API[API REST NΛCO]
  W --> API
  API --> PG[(PostgreSQL)]
  API <--> RD[(Redis)]
  API -->|upload/exclusão via StoragePort| CLD[Cloudinary: mídia + CDN]
  M -->|GET de imagem| CLD
  W -->|GET de imagem| CLD
```

O mobile e o web não acessam o banco nem as APIs administrativas do Cloudinary. Eles recebem da API a URL pública versionada e carregam a imagem pela CDN gerenciada. Toda operação de upload, substituição ou exclusão passa pela API; `api_secret` fica exclusivamente no backend/ambiente seguro.

6.3 Diagrama de contêineres e responsabilidades

Contêiner	Tecnologia	Responsabilidade
Mobile cliente	React Native + TypeScript	Descoberta, catálogo, carrinho local, checkout, histórico, acompanhamento e reviews.
Web gestão	React + TypeScript	Restaurantes, horários, categorias, produtos, pedidos, inbox e dashboard.
API	Node.js 26 LTS, Express, TypeScript strict	Autenticação, autorização, validação, domínio, contratos e integração.
Banco relacional	PostgreSQL	Fonte de verdade, constraints, transações, snapshots e auditoria.

Contêiner	Tecnologia	Responsabilidade
Cache/rate-limit	Redis	Cache-aside público, contadores e limites; não fonte de verdade.
Armazenamento de imagens	Cloudinary via `StoragePort`	Backend controla upload/processamento/exclusão; Cloudinary armazena e distribui imagens públicas.
CI	GitHub Actions	Lint, typecheck, testes, build e verificação de migrations.

6.4 Camadas internas

1. **Routes:** composição de métodos, paths, middlewares e controllers; sem regras de domínio.
2. **Controllers:** tradução HTTP ⇄ casos de uso; validam envelope/DTO e códigos HTTP; não orquestram SQL diretamente.
3. **Application/Services:** casos de uso, autorização contextual, transações e coordenação de domínio.
4. **Domain:** entidades/valores, invariantes, máquina de estados, cálculo monetário, normalização, políticas de autorização.
5. **Repositories:** abstração de persistência; implementação Prisma; consultas específicas e transações.
6. **Schemas/DTOs:** Zod para entrada e saída; contratos OpenAPI; mapeamento explícito camelCase ⇄ modelo relacional.
7. **Infrastructure:** PostgreSQL/Prisma, Redis, Cloudinary (adapter `StoragePort`), relógio, UUID, hashing, JWT, logging e tracing.

Direção de dependência: interface → aplicação → domínio; infraestrutura implementa portas definidas pelas camadas internas. O domínio não importa Express, Prisma, Redis nem SDK do Cloudinary.

6.5 Módulos de domínio

Módulo	Responsabilidades	Dependências permitidas principais
`auth`	Cadastro, login, refresh, logout, sessões, hashing, tokens.	users, sessions, security infra
`users`	Perfil e estado de usuário.	roles
`roles`	UserRole, solicitações, decisões, auditoria.	users, restaurants (transferência)
`restaurants`	Cadastro, propriedade, publicação, horário, timezone.	users, orders (bloqueio de despublicação)
`categories`	Categorias por restaurante e ordenação.	restaurants
`products`	ProductBase, RestaurantProduct, preço, stock, mídia, normalização.	restaurants, categories, storage
`orders`	Carrinho não persistido, criação, idempotência, snapshots, estado, eventos, stock, views.	users, restaurants, products
`reviews`	Review PRODUCT_ONLY e validação de ownership/conclusão.	orders/products
`dashboard`	Agregações de vendas concluídas e reviews.	restaurants, orders, reviews
`admin`	Casos de uso privilegiados e trilha de auditoria.	roles, restaurants, users
`shared`	Erros, paginação, money, clock, IDs, logging, config.	sem dependência dos módulos de domínio

A dependência entre módulos deve ocorrer por interfaces/casos de uso exportados, não por importação de repositórios internos de outro

módulo.

6.6 Estrutura de repositório recomendada

```
naco/
├── apps/
│   ├── api/                    # Express + TypeScript
│   ├── mobile/                 # React Native
│   └── web/                    # React web
├── packages/
│   ├── contracts/              # Schemas compartilháveis, tipos públicos, OpenAPI
│   └── config/                 # Configurações TypeScript/lint compartilhadas
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── api/
│   ├── testing/
│   └── operations/
├── infra/
│   ├── docker/
│   └── scripts/
├── .github/workflows/
├── docker-compose.yml
├── .env.example
└── README.md
```

Dentro de cada módulo da API, manter `domain`, `application`, `infrastructure`, `http` e `tests` quando o tamanho justificar. Não criar camadas vazias só por padronização: começar simples, mas manter fronteiras claras.

7. Stack tecnológica e justificativas

Camada	Decisão do adendo	Uso e observações
Runtime backend	Node.js 26 LTS	Fixar versão exata suportada no ambiente e CI; não usar Alpha/Current em produção.
Backend HTTP	Express.js	API REST; middlewares explícitos e composição modular.

Camada	Decisão do adendo	Uso e observações
Linguagem	TypeScript `strict`	Tipagem estrita e contratos internos.
Banco	PostgreSQL	ACID, constraints, índices, transações e JSONB para snapshot de endereço quando apropriado.
ORM/migrations	Prisma	Acesso tipado e migrações versionadas; SQL explícito para índices parciais/triggers quando Prisma não expressar constraint necessária.
Cache/rate limit	Redis	Cache-aside e limitadores; política para indisponibilidade deve ser definida.
Mobile	React Native + TypeScript + React Navigation + Axios	Aplicativo cliente; refresh seguro via Keychain/Keystore.
Web	React + TypeScript + Axios	Painel do responsável e administração.
Validação	Zod	Validar input no limite HTTP; validar invariantes no domínio.
Contratos	OpenAPI	Fonte publicada dos paths, schemas, autenticação, erros e exemplos.
Imagens	Cloudinary atrás de `StoragePort`	Fornecedor aprovado para MVP; pipeline, exclusão best effort e operações persistentes definidos em `05_ARMAZENA-MENTO_E_PROCESSAMENTO_DE_IMAGENS.md`.

Camada	Decisão do adendo	Uso e observações
Testes	Vitest, Supertest, React Testing Library, Appium, k6	Unitário, API, UI, mobile E2E e carga.
Dev local	Docker Compose	PostgreSQL e Redis; seeds somente para desenvolvimento/testes.
CI	GitHub Actions	lint, typecheck, unit, integration, migrations, build, security checks.

7.1 Redis: responsabilidade e limites

- Cache-aside apenas para consultas públicas de restaurantes e catálogo.
- TTL inicial de 60 segundos.
- Invalidar após alterações relevantes em restaurante, produto ou categoria.
- Não cachear estoque, permissões, pedidos ou sessões como fonte de verdade.
- Rate limiting por janelas deslizantes/token bucket.
- Definir comportamento explícito na queda de Redis: não degradar silenciosamente a proteção de endpoints sensíveis. A política fail-closed/degraded com fallback limitado precisa ser aprovada e testada.

7.2 Armazenamento de imagens

Criar interface `StoragePort` (`upload`, `delete`, `getPublicUrl`, `getUsage`) com adapter Cloudinary. O backend é o único componente com credenciais administrativas e controla upload, substituição e exclusão. Persistir chave lógica, `asset_id`, `public_id` e `version`; o adapter gera URL versionada. Segredos ficam em configuração segura, nunca no cliente. Cloudinary foi escolhido para o MVP no plano Free sem cartão.

O upload aceita JPEG/PNG/WebP estáticos, até 5 MB e 40 MP, com validação de extensão, MIME, magic bytes e decodificação. O pipeline remove EXIF, converte para sRGB, preserva transparência, limita a imagem a 1600×1600 sem upscale e gera WebP com qualidade configurável. A imagem anterior só é substituída após upload concluído e atualização condicional. Cada chave é imutável e única por opera-

ção; URLs de mídia usam public IDs e versões imutáveis; não assumir TTL de um ano no CDN padrão do Cloudinary.

Upload, eventos append-only, reconciliação e exclusão são operações persistidas. No Cloudinary, `destroy` com `invalidate=true` solicita exclusão e invalidação na mesma chamada; a resposta não comprova a propagação global do purge e não há polling padrão por ID. URLs versionadas, `public_id` imutável, retries transitórios e reconciliação da existência do asset seguem `05_ARMAZENAMENTO_E_PROCESSAMENTO_DE_IMAGENS.md`. O monitoramento interno bloqueia novos uploads ao atingir o limiar configurado; nenhuma cobrança/upgrade é permitido sem autorização.

8. Modelo de domínio e persistência

8.1 Entidades centrais

Entidade	Finalidade	Campos/atributos documentados ou derivados
`User`	Conta única	id, name, email único, password_hash, status, timestamps.
`UserRole`	Papéis por conta	user_id, role; PK composta; ADMIN exclusivo/único.
`RoleRequest`	Solicitação RESPONSAVEL	id, user_id, status, created_at, decided_at, decided_by, decision_reason opcional.
`RoleAuditEvent`	Auditoria administrativa	id, actor_user_id, target_user_id, action, reason, metadata JSONB, created_at.

Entidade	Finalidade	Campos/atributos documentados ou derivados
`Restaurant`	Estabelecimento	responsável, nomes, CNPJ único, contato/endereço, cozinha, timezone, opening_hours, logo/cover, publication_status.
`ProductBase`	Identidade compartilhável de produto	id, name, description, timestamps.
`Category`	Categoria própria do restaurante	id, restaurant_id, name, description, display_order, status.
`RestaurantProduct`	Oferta comercial específica	id, restaurant_id, product_base_id, category_id, display_name, normalized_name, price, stock_quantity, `storage_key` opcional, `provider_asset_id`, `provider_public_id`, `provider_version`, status, timestamps.
`Order`	Pedido simulado	id, customer_id, restaurant_id, status, total, delivery address/name/email snapshots, timestamps.
`OrderItem`	Linha histórica	id, order_id, restaurant_product_id, product_name snapshot, unit_price, quantity, subtotal.

Entidade	Finalidade	Campos/atributos documentados ou derivados
`OrderEvent`	Histórico de transição	id, order_id, previous_status, new_status, changed_by, created_at.
`OrderView`	Primeiro acesso do responsável	order_id, responsible_user_id, first_viewed_at; unique composto.
`Review`	Avaliação de produto	id, order_item_id único, rating, comment, timestamps.
`IdempotencyRecord`	Estado da intenção de checkout	user_id, key, payload_hash, processing/result state, HTTP result, order reference, expiry/tombstone timestamps.
`StorageUploadOperation`	Operação persistente de upload	id, restaurant_product_id, storage_key, provider_asset_id, provider_public_id, provider_version, estado, timestamps e dados de tentativas/reconciliação.
`StorageUploadOperationEvent`	Histórico de upload	id, upload_operation_id, event_type, details JSONB, actor_type/actor_user_id, process_instance_id, created_at; append-only.

Entidade	Finalidade	Campos/atributos documentados ou derivados
`StorageDeletionOperation`	Exclusão/invalidação best effort de asset Cloudinary	storage_key único, asset_id/public_id, tentativas, resposta do provider e reconciliação da existência do asset.

`RoleRequest`, `RoleAuditEvent` e `IdempotencyRecord` têm necessidade funcional e campos lógicos definidos no adendo; o schema físico final, constraints, índices e migrações ainda precisam ser consolidados. `OrderView` também precisa de schema físico.

8.2 Relacionamentos

```
erDiagram
  USER ||--o{ USER_ROLE : has
  USER ||--o{ RESTAURANT : manages
  USER ||--o{ ORDER : creates
  RESTAURANT ||--o{ CATEGORY : owns
  RESTAURANT ||--o{ RESTAURANT_PRODUCT : offers
  PRODUCT_BASE ||--o{ RESTAURANT_PRODUCT : identifies
  CATEGORY ||--o{ RESTAURANT_PRODUCT : classifies
  RESTAURANT ||--o{ ORDER : receives
  ORDER ||--|{ ORDER_ITEM : contains
  RESTAURANT_PRODUCT ||--o{ ORDER_ITEM : snapshots_from
  ORDER ||--o{ ORDER_EVENT : records
  ORDER_ITEM ||--o| REVIEW : reviewed_as
  ORDER ||--o{ ORDER_VIEW : viewed_by
  USER ||--o{ ROLE_REQUEST : requests
  USER ||--o{ ROLE_AUDIT_EVENT : acts_or_is_target
```

8.3 Restrições e índices obrigatórios/conceituais

- `User.email` globalmente único.
- `UserRole(user_id, role)` chave composta.
- Uma única conta ADMIN global; ADMIN não combina com CLIENTE/RESPONSAVEL.
- `Restaurant.cnpj` único.
- `RestaurantProduct.price > 0`; `stock_quantity >= 0`.
- Constraint composta garantindo que category e offer pertencem ao mesmo restaurante.

- `RestaurantProduct(restaurant_id, normalized_name)` único, inclusive desativados.
- `OrderItem.quantity` entre 1 e 20.
- `UNIQUE(order_id, restaurant_product_id)`.
- Pedido deve ter ao menos um item; assegurar no caso de uso/transação.
- Subtotal = unit price × quantity; total = soma dos subtotais.
- `Review.order_item_id` único; rating 1–5.
- `IdempotencyRecord(user_id, key)` único e tombstone permanente.
- `OrderView(order_id, responsible_user_id)` único.
- RoleRequest: uma solicitação pendente por usuário via índice parcial (conforme adendo).
- Constraints de estoque e status devem proteger invariantes mesmo sob concorrência.

8.4 Dinheiro e precisão

- Moeda única: BRL.
- Persistência: `DECIMAL(12,2)`.
- API: strings decimais, por exemplo `"19.90"`.
- Domínio: Decimal ou inteiro em centavos; nunca `float` de ponto flutuante binário.
- Rejeitar mais de duas casas decimais com HTTP 422.
- `subtotal = unitPrice × quantity`; `total = soma(subtotal)`.
- Não há taxa de entrega nem pagamento no MVP.

8.5 Horários e timezone

- `Restaurant.timezone` usa identificador IANA, inicialmente `America/Sao_Paulo` até ajuste conforme localização.
- Grade semanal ISO: 1=segunda a 7=domingo.
- Cada dia aceita zero ou mais intervalos; ausência de intervalo significa fechado.
- Horas no formato HH:mm.
- Intervalo com `closesAt < opensAt` atravessa meia-noite; é interpretado como continuação do dia de abertura.
- `closesAt == opensAt` é inválido.
- Intervalos sobrepostos são proibidos.
- Grade ausente ou inválida significa fechado (fail-safe).
- Avaliação usa hora local do restaurante e deve considerar intervalos iniciados no dia anterior.
- Feriados, exceções e fechamento especial ficam fora do MVP.

Validação adicional recomendada: verificar colisões entre dias

adjacentes (ex.: segunda 22:00–02:00 e terça 01:00–05:00), não apenas sobreposição dentro da mesma lista diária.

8.6 Normalização de nome

Algoritmo servidor: Unicode NFKD → remover marcas diacríticas → trim → colapsar espaços consecutivos → lowercase independente de locale. Persistir `normalized_name` ou derivá-lo consistentemente. Unicidade composta por restaurante. Não tentar equivalência de homógrafos entre alfabetos diferentes. Atualização dos dois campos deve ser atômica.

9. Arquitetura de pedidos, estoque e idempotência

9.1 Estados canônicos

`CRIADO` → `CONFIRMADO_RESTAURANTE` → `EM_PREPARO` → `SAIU_PARA_ENTREGA` → `CONCLUIDO`.

Saídas alternativas: `CRIADO` → `RECUSADO`; cancelamentos conforme matriz. Persistência/API usam `CONCLUIDO` sem acento; a UI pode exibir "Concluído".

9.2 Máquina de estados e autorização

Toda transição deve verificar simultaneamente: 1. usuário autenticado e sessão ativa; 2. papel exigido; 3. ownership/escopo sobre o restaurante ou pedido; 4. estado atual real no banco; 5. destino permitido pela máquina de estados; 6. pré-condições adicionais (estoque, elegibilidade de cancelamento).

Usar update condicional ou lock transacional para garantir que duas requisições concorrentes não confirmem/cancelem/avancem o mesmo pedido. Mudança de status e `OrderEvent` pertencem à mesma transação. Em conflito, retornar `409 ORDER_STATE_CONFLICT`, sem persistência parcial.

9.3 Estoque

Criação: validar estoque suficiente como leitura de viabilidade, mas não reservar nem debitar. Como não há reserva, outra compra pode consumir estoque antes da confirmação; a confirmação deve revalidar.

Confirmação: bloquear linhas de ofertas em ordem determinística por ID, validar todas as quantidades, aplicar débitos, mudar status e inserir evento em transação única. Se um item falhar, rollback total e pedido permanece CRIADO.

Cancelamento: se a baixa já ocorreu, transicionar, criar evento e restituir exatamente uma vez na mesma transação. Estados que não tiveram baixa não podem gerar reposição.

9.4 Idempotência de criação

- Header obrigatório `Idempotency-Key` UUID v4.
- Unicidade por `(user_id, key)`.
- Hash canônico inclui `restaurantId`, endereço normalizado e itens ordenados por `restaurantProductId` com quantidade e preço esperado.
- Mesma chave + mesmo hash retorna resultado original.
- Mesma chave + payload diferente: 409 `IDEMPOTENCY_KEY_REUSED`.
- Preço esperado divergente: 409 `PRICE_CHANGED`; sem pedido criado.
- Estado completo por 24h; depois tombstone mínimo permanente; chave não reutilizável.
- Concorrente aguarda até 5s sem manter transação/lock longo; depois 202 `PROCESSING` com URL de consulta; primeira solicitação continua.
- `GET /api/orders/idempotency/{key}` verifica usuário dono; desconhecida/outro usuário = 404; tombstone próprio expirado = 410; processing/concluída = 200.
- Falha antes de commit reverte registro, pedido e itens integralmente.

9.5 Compatibilização QU-60 — resolução consolidada

A QU-60 está resolvida pelo adendo complementar, sem alterar a regra funcional de estoque. A atomicidade tem duas fronteiras: - **Criação:** registro idempotente + Order + OrderItems, na mesma transação, sem reservar ou alterar estoque. - **Confirmação/cancelamento:** estoque + status + OrderEvent, com baixa ou reposição atômica na mesma transação.

A criação não reduz estoque; a confirmação revalida disponibilidade e debita; cancelamento elegível após confirmação restaura somente uma vez.

Não antecipar débito de estoque para criação. Testar falhas an-

tes/depois do commit e repetição de chave para provar ausência de duplicidade.

10. Contratos de API

10.1 Padrões gerais

- Base path: `/api`.
- REST/HTTP; JSON `camelCase`; banco pode ser `snake_case`.
- Datas UTC ISO 8601 com `Z`; horários de restaurante são hora local + timezone.
- Campo canônico de estado: `status`.
- Oferta externa: `restaurantProductId`; catálogo: `price`; snapshot de pedido: `unitPrice`.
- Paginação: `page=1`, `limit=20`, máximo 100; resposta com `items`, `page`, `limit`, `totalItems`, `totalPages`.
- O cliente nunca envia `customerId`/`responsibleUserId` como autoridade; o backend obtém identidade da sessão.
- DTOs são schemas dedicados; não serializar entidades Prisma diretamente.

10.2 Inventário de rotas orientador

Método	Rota	Acesso	Finalidade
POST	`/api/auth/register`	Público	Cadastro CLIENTE.
POST	`/api/auth/login`	Público	Iniciar sessão.
POST	`/api/auth/refresh`	Cookie/refresh válido	Renovar e rotacionar refresh.
POST	`/api/auth/logout`	Sessão	Revogar sessão.
GET/PATCH	`/api/me`	Autenticado	Consultar/alterar nome próprio.
GET	`/api/restaurants`	Público	Listar restaurantes públicos.
GET	`/api/restaurants/{id}`	Público	Detalhe público.
GET	`/api/restaurants/{id}/products`	Público	Catálogo.
POST	`/api/restaurants`	RESPONSAVEL	Criar restaurante.

Método	Rota	Acesso	Finalidade
GET	`/api/me/restaurants`	RESPONSAVEL	Restaurantes gerenciados.
GET	`/api/restaurants/{id}/management`	Responsável do restaurante	Dados para gestão.
PATCH	`/api/restaurants/{id}`	Responsável do restaurante	Atualizar dados/horários.
PATCH	`/api/restaurants/{id}/publication`	Responsável do restaurante	Publicar/despublicar.
POST	`/api/restaurants/{id}/products`	Responsável do restaurante	Criar oferta.
PATCH	`/api/products/{id}`	Responsável do restaurante	Atualizar oferta.
DELETE	`/api/products/{id}`	Responsável do restaurante	Desativar/restringir exclusão.
POST	`/api/orders/create`	CLIENTE	Criar pedido; Idempotency-Key obrigatório.
GET	`/api/orders`	CLIENTE	Histórico próprio.
GET	`/api/restaurants/{restaurantId}/orders`	Responsável do restaurante	Lista de pedidos.
GET	`/api/me/orders/inbox`	RESPONSAVEL	Inbox consolidada.
GET	`/api/orders/{orderId}`	Titular ou operador autorizado	Detalhes e eventos.
POST	`/api/orders/{orderId}/views`	Responsável autorizado	Registrar primeira visualização; idempotente por pedido/responsável.
PATCH	`/api/orders/{orderId}/status`	Autorizados	Único contrato de transição.
GET	`/api/orders/idempotency/{key}`	Titular da chave	Consultar estado idempotente.
GET	`/api/admin/storage/upload-operations`	ADMIN	Listar operações de upload com filtros por estado/período.

Método	Rota	Acesso	Finalidade
GET	`/api/admin/storage/upload-operations/{id}`	ADMIN	Consultar detalhes e histórico de uma operação de upload.
POST	`/api/order-items/{orderItemId}/reviews`	Cliente dono do pedido	Avaliar produto após CONCLUIDO.
GET	`/api/products/{productId}/reviews`	Público, sem autenticação	Consultar avaliações públicas paginadas; autor exibido como “Cliente verificado”.
GET	`/api/restaurants/{restaurantId}`	Responsável autorizado	Dashboard do restaurante indicado; ownership validado.
POST/GET	`/api/me/role-requests`	Autenticado	Criar/listar solicitações próprias.
GET	`/api/admin/role-requests?status=PENDING`	ADMIN	Listar solicitações.
PATCH	`/api/admin/role-requests/{id}`	ADMIN	Aprovar/rejeitar.
PATCH	`/api/admin/restaurants/{id}/responsible`	ADMIN	Transferir responsável.
DELETE	`/api/admin/users/{userId}/roles/RESPONSAVEL`	ADMIN	Remover papel após transferências.

A visualização é uma escrita explícita em `POST /api/orders/{orderId}/views`; `GET /api/orders/{orderId}` permanece uma leitura pura. O `POST` preserva o primeiro `first_viewed_at` por responsável.

10.3 Exemplo de criação de pedido

```
POST /api/orders/create
Authorization: Bearer <access-token>
```

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json
```

```
{
  "restaurantId": "rest_123",
  "deliveryAddress": {
    "recipientName": "Cliente de teste",
    "postalCode": "99999-000",
    "street": "Rua Exemplo",
    "number": "100",
    "complement": null,
    "neighborhood": "Centro",
    "city": "Tapejara",
    "state": "RS"
  },
  "items": [
    {
      "restaurantProductId": "offer_456",
      "quantity": 2,
      "expectedUnitPrice": "19.90"
    }
  ]
}
```

Identidade do cliente, nome do produto, preço final, subtotal e total são derivados pelo backend. O endereço é validado e persistido como snapshot imutável.

10.4 Envelope de erro

```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Não há estoque suficiente para confirmar este pedido.",
    "details": [],
    "requestId": "req_..."
  }
}
```

	HTTP	Uso previsto
	200	Consulta/alteração concluída.
	201	Recurso criado.
	202	Intenção idempotente em processamento.

	HTTP	Uso previsto
	400	JSON malformado, header obrigatório ausente ou sintaxe inválida.
	401	Não autenticado/sessão inválida ou expirada sem renovação possível.
	403	Autenticado sem papel global exigido.
	404	Recurso inexistente ou fora do escopo, sem revelar existência.
	409	Conflito de domínio/concorrência/idempotência/preço/estoque.
	410	Tombstone expirado pertencente ao usuário.
	422	Documento válido sintaticamente, mas semanticamente inválido.
	429	Rate limit.
	500	Erro inesperado; sem internals.
	503	Dependência temporariamente indisponível.

Resolver antes da API freeze os códigos legados divergentes e padronizar `error.code` simbólico estável.

11. Segurança e privacidade

11.1 Senhas e tokens

- Senha: 12–128 caracteres; Unicode e espaços aceitos; sem regras artificiais de composição.
- Hash: Argon2id com baseline `m=19456 KiB`, `t=2`, `p=1`; calibrar sem reduzir abaixo do baseline sem revisão. Salt pela biblioteca.
- Access JWT HS256, 15 minutos, segredo aleatório de pelo menos 256 bits em secret manager/ambiente protegido.
- Refresh opaco, aleatório, rotativo, até 30 dias; banco armazena hash, sessão, usuário, expiração e revogação.
- Reutilização de refresh antigo revoga a família de sessão.

- API verifica sessão ativa; logout revoga; alteração de status/papel reflete imediatamente na autorização.

11.2 Armazenamento no cliente

- Mobile: access token em memória; refresh em Keychain/Keystore com biblioteca segura.
- Web: refresh em cookie HttpOnly, Secure, SameSite=Lax; access token em memória.
- Proibido armazenar tokens em localStorage/sessionStorage.
- Web mutável precisa de proteção CSRF e CSP; prevenir XSS; usar HTTPS.
- Logout limpa carrinho e dados temporários locais.

11.3 Autorização

- Middleware autentica sessão, mas serviços verificam autorização contextual.
- Verificar papel, propriedade do restaurante, ownership do pedido, estado do usuário e estado da sessão em cada ação.
- Nunca confiar em role, userId, restaurantId ou owner enviados pelo frontend.
- Para recurso alheio, preferir 404 quando a regra de privacidade exige não revelar existência.
- ADMIN único, exclusivo e auditado.

11.4 Rate limiting e defesa

- Login: 5 falhas por conta/15 min; 30 tentativas por IP/15 min; atraso progressivo e mensagem genérica.
- Cadastro: 5 tentativas/IP/15 min.
- Geral: 120 req/min por usuário autenticado; limites separados para ações sensíveis.
- Redis sliding window/token bucket; 429 e Retry-After.
- CORS limitado a origens conhecidas, TLS, Helmet, CSP, validação Zod, queries parametrizadas, upload limitado, secret scanning e dependency checks no CI.
- Upload de imagem: somente backend acessa o bucket; validar extensão, MIME, magic bytes, decodificação, 5 MB, 40 MP e formato estático; não aceitar URL arbitrária como prova de arquivo seguro.
- Remover EXIF, normalizar para sRGB/WebP e não expor credenciais/URLs assinadas; arquivos temporários devem ser limpos e a concorrência de conversões limitada.

11.5 Dados que não podem ir aos logs

Senha, access token, refresh token, Idempotency-Key completa, endereço completo, payload sensível, dados pessoais não necessários e segredos. Usar requestId/correlationId e identificadores não sensíveis/pseudonimizados quando necessário.

11.6 Auditoria e retenção

- `RoleAuditEvent` append-only: actor, target, action, reason, metadata e created_at.
- Aplicação deve impedir alteração/remoção; reforçar com privilégios de banco e/ou trigger onde apropriado.
- Logs de aplicação: 90 dias como baseline.
- Auditoria: 24 meses como baseline, sujeita a revisão de necessidade/privacidade.
- Dados pessoais exigem política de retenção/eliminação antes de produção. A presença desses controles não declara conformidade jurídica automática.
- MFA fora do MVP.

12. Frontend mobile e web

12.1 Mobile — cliente

Módulos/telas esperados: autenticação; home/explorar restaurantes; filtros/busca; detalhe do restaurante; catálogo por categoria; detalhe do produto; carrinho; checkout/endereço; confirmação de envio; histórico; detalhe/acompanhamento do pedido; avaliações de item concluído; perfil.

Estado local: sessão; carrinho associado ao usuário; restaurante atual do carrinho; itens/quantidades; totais calculados com valores decimais seguros; estado de checkout e chave idempotente; cache visual não autoritativo.

Regra do carrinho: um restaurante por vez; ao mudar de restaurante, não misturar produtos. A política de UX ao tentar adicionar de outro restaurante deve preservar RN-01 e explicar a ação necessária, sem criar pedido multirrestaurante.

Checkout resiliente: desabilitar submissão duplicada; manter a mesma chave em retries da mesma intenção; em timeout, consultar a chave; se preço mudou, pedir revisão e nova chave; não mostrar sucesso até confirmação/consulta positiva.

12.2 Web — responsável/admin

Áreas: seleção de restaurante/contexto; resumo; cadastro/edição do estabelecimento; horários/timezone; publicação; categorias; catálogo; disponibilidade/estoque; pedidos por restaurante; inbox consolidada; detalhe do pedido; ações de estado; contadores não lidos; dashboard; administração de solicitações/transferências/auditoria.

Pedidos: polling a cada 15 segundos enquanto visível; pausar em background/aba oculta; atualização imediata no retorno. Preservar filtros, pesquisa e página ao voltar dos detalhes. Após ação concorrente/falha, consultar estado atual antes de habilitar nova tentativa.

Acessibilidade: usar WCAG 2.2 AA como referência para web; foco visível, labels, contraste, navegação por teclado, erros associados aos campos, estados comunicados a leitores de tela. No mobile: rótulos acessíveis, ordem coerente, contraste e texto ampliável.

13. Segurança operacional e integração

13.1 Política de retries (RNF-02 operacionalizado)

- GET idempotente: até 3 tentativas totais em falha transitória de rede/timeout/429/502/503/504; backoff exponencial com jitter e respeito a Retry-After.
- 401: uma tentativa de refresh e uma repetição da chamada original.
- Não retry automático em 400/403/404/409/422.
- Criação de pedido: até 3 tentativas com a mesma chave; em timeout consultar chave primeiro.
- Transições de pedido: nunca retry cego; consultar estado e reapresentar ações válidas.
- Em qualquer falha, mostrar mensagem útil e nunca simular sucesso.

13.2 Disponibilidade, backup e recuperação

- Sem SLA contratual no MVP.
- Meta interna inicial pós-deploy: 99,5% de disponibilidade mensal, excluindo manutenção planejada.
- Backup diário criptografado do PostgreSQL, retenção 7 dias.
- Snapshot antes de migrations relevantes.
- RPO inicial até 24 horas; RTO inicial até 8 horas.
- Teste de restauração mensal, com evidência registrada.

13.3 Observabilidade

- Logs estruturados com requestId/correlationId.
- Métricas de latência, taxa de requisição, erro, CPU, RAM, conexões DB, Redis/cache, fila interna se existir e falhas externas.
- Pipeline de imagens: sucesso/falha, duração, retries, volume processado, consumo de recursos e operações problemáticas; eventos de upload append-only e operações de exclusão/invalidação best effort do Cloudinary persistidas.
- Tracing OpenTelemetry para requisição e chamadas ao banco.
- Alertas iniciais: erro técnico >1% por 5 min; P95 acima do limite por 5 min; banco/Redis indisponível; backup falhou.
- Logs técnicos 90 dias; trilha administrativa 24 meses, sujeita à revisão de privacidade.
- Health/readiness não expõe segredo, stack trace ou configuração sensível.

14. Estratégia de testes e qualidade

14.1 Pirâmide de testes

Nível	Ferramenta	Escopo
Unitário	Vitest	Regras, estado, dinheiro, normalização, autorização, horários, validações.
Integração backend	Vitest + PostgreSQL real/Testcontainer	Migrations, constraints, transações, idempotência, estoque, sessões.
API	Supertest	Auth, ownership, status, DTO, paginação, erros, headers.
UI web/mobile	React Testing Library	Componentes, estados de erro/loading, acessibilidade básica, formulários.
E2E mobile	Appium	Jornada real do cliente em dispositivo/emulador.

Nível	Ferramenta	Escopo
Carga/concor.	Grafana k6	RNF-01, estoque concorrente, latência, RPS e taxa de erro.
Exploratórios	Postman	Inspeção manual e smoke, sem substituir testes automatizados.

14.2 Fluxos E2E obrigatórios

1. Cadastro, login, refresh, logout.
2. RESPONSAVEL solicita papel; ADMIN aprova; usuário cadastra restaurante.
3. Responsável cria categoria e oferta; catálogo público apresenta oferta.
4. Cliente encontra restaurante, adiciona produtos e faz checkout.
5. Mesma chave repetida retorna o pedido original; chave com payload diferente recebe 409.
6. Responsável recebe, confirma, prepara e avança pedido.
7. Estoque baixa somente na confirmação; cancelamento elegível restaura uma vez.
8. Cliente consulta histórico e avalia item após CONCLUIDO.
9. Inbox consolidada separa restaurantes e contadores por responsável.
10. Dashboard calcula produtos vendidos/reviews no período.

14.3 Testes críticos de segurança e concorrência

- Acesso cruzado entre clientes e entre restaurantes.
- Spoof de role/userId/restaurantId no cliente.
- Token revogado, sessão encerrada e refresh reutilizado.
- Rate limit por conta/IP/usuário.
- SQL injection, XSS/CSRF web e upload malicioso.
- Vazamento de secrets/PII nos logs.
- Mesma chave idempotente simultânea; mesma chave com payload diferente; timeout antes/depois do commit.
- Duas confirmações concorrentes; confirmação × cancelamento; cancelamento duplicado; duas compras disputando o mesmo estoque.
- Corrida entre remoção de RESPONSAVEL e requisição já em andamento.

14.4 Critérios de qualidade e gate de release

- CI executa lint, typecheck, unitários, integração, build e validação de migrations.
- Meta de cobertura ≥80% em serviços/regras de domínio; cobertura global isolada não prova qualidade.
- Zero bug crítico/alto conhecido nos fluxos de autenticação, autorização, pedido, estoque e idempotência.
- Smoke test em staging aprovado.
- Testes de migração/rollback e restauração com evidência.
- RNF-01 só aprovado após executar campanhas e critérios completos; documentação não substitui medição.

15. RNF-01 — Desempenho e integridade sob carga

15.1 Operações oficiais

1. Listar restaurantes.
2. Consultar detalhes de restaurante.
3. Listar produtos.
4. Consultar detalhes de produto.
5. Login/autenticação.
6. Criar pedido.
7. Consultar pedidos do cliente.
8. Consultar pedidos recebidos pelo restaurante.
9. Consultar painel de gerenciamento.

A operação 8 depende de contrato coerente de ator/método/rota. Fechar antes da campanha.

15.2 Limites oficiais

Classe	P95	P99
Operações comuns	≤2 s	≤4 s
Criação de pedido	≤3 s	≤6 s

Taxa de erro técnico máxima: 1%. Erros de negócio esperados como 409 de estoque não contam como erro técnico, conforme definição do requisito; outras classes 409 devem ser classificadas/documentadas para evitar métricas ambíguas.

15.3 Carga sustentada

- Ramp-up progressivo até 1.000 usuários virtuais simultâneos: 10 minutos.
- Sustentar 1.000 VUs por 60 minutos adicionais.
- Trecho oficial por execução: mínimo 70 minutos, fora preparação/aquecimento/encerramento.
- Sessões persistentes e pool de contas com papéis apropriados; não autenticar a cada requisição.
- Dataset sintético e isolado.

15.4 Matriz quantitativa de homologação

Campanha	Desenho	Execuções
Carga	3 volumes × 2 condições de cache × 5 execuções	30
Overhead	30 pares × 2 configurações de instrumentação	60
Concorrência de estoque	4 níveis × 10 repetições	40
Total		**130**

Cache frio: limpar previamente e registrar misses/aquecimento. Cache aquecido: pré-carregar amostra e confirmar hit rate. Antes da medição, estabilizar por 5 min com CPU/RAM/conexões DB sem tendência e dentro de ±5% da linha base; sem requisições pendentes nem filas de infraestrutura. Alternar instrumentação completa/mínima e não subtrair overhead dos resultados oficiais. Percentis de concorrência calculados sobre latências individuais agregadas nas 10 repetições de cada nível, com amostra reportada e limitação do P99 reconhecida.

15.5 Evidências obrigatórias

Guardar versão de código, commit, configuração, hardware/infra, dataset, scripts k6, logs, métricas, latências individuais, percentis, RPS, erros, recursos, condição de cache, modo de instrumentação e relatório de cada execução. Campanha aprovada só após todos critérios e cenários QU correspondentes executados e avaliados.

16. CI/CD, ambientes e release

16.1 Ambientes

- **Local:** Docker Compose, dados sintéticos, `.env` local não versionado.

- **CI:** serviços efêmeros PostgreSQL/Redis; migrations e testes repetíveis.
- **Staging:** integração semelhante à produção com dados sintéticos; smoke, segurança e ensaio de release.
- **Produção/demonstração pública:** secrets protegidos, TLS, backup, monitoramento, acesso administrativo controlado.

16.2 Pipeline sugerido

1. Checkout e configuração da versão Node fixada.
2. Instalação reproduzível com lockfile.
3. Lint e typecheck.
4. Testes unitários.
5. Inicializar PostgreSQL/Redis de teste.
6. Executar migrations e testes de integração.
7. Build backend/mobile/web.
8. Verificação de dependências e secrets.
9. Artefatos e relatório de cobertura.
10. Deploy staging e smoke tests via etapa protegida.
11. Aprovação manual antes de produção, se houver ambiente público.

16.3 Migrations

- Toda alteração de schema tem migration versionada.
- Migration deve ser testada em banco vazio e em cópia representativa.
- Criar snapshot antes de mudança relevante.
- Preferir migração compatível (expand/contract) para evitar incompatibilidade entre versão antiga e nova durante rollout.
- Não usar seed administrativo público nem credenciais padrão versionadas.

17. Gestão de riscos técnicos

Risco	Impacto	Mitigação/teste
Conflito de leitura OrderView e GET puro	Efeito colateral em método de consulta	Separar POST de visualização; teste de idempotência; corrigir OpenAPI.

Risco	Impacto	Mitigação/teste
Conta com papéis múltiplos em clientes distintos	UX pode confundir qual área está disponível	Não duplicar conta nem introduzir seletor de identidade no login; mobile exige CLIENTE, web exige RESPONSAVEL, e backend valida papel/ownership em cada operação. Cobrir guards na implementação.
Dashboard para responsável com vários restaurantes	Interface pode exibir métricas do restaurante errado	`restaurantId` obrigatório no endpoint; UI deve selecionar explicitamente o restaurante e validar ownership no backend.
Concorrência de estoque	Sob venda ou débito duplicado	Lock ordenado, update condicional, transação e teste concorrente no PostgreSQL.
Retry de checkout	Pedido duplicado	Idempotency-Key persistente e consulta após timeout.
Redis indisponível	Rate limit/cache inconsistente	Definir fail-closed/degraded por endpoint e testar falha.
Mudança de catálogo	Histórico corrompido	Snapshot e teste de imutabilidade.
Horário entre dias	Restaurante aberto/fechado errado	Validar adjacência e timezone IANA.
Perda de responsável	Restaurante órfão/inacessível	Transferência obrigatória antes de remoção.
Segredo/PII em logs	Exposição de dados	Sanitização estruturada, testes de logs e secret scanning.

Risco	Impacto	Mitigação/teste
RNF-01 sem ambiente representativo	Conclusão inválida de desempenho	Documentar ambiente/dataset, repetir execuções e não extrapolar resultados.

18. Itens remanescentes para contrato, implementação e operação

As decisões de produto e arquitetura do MVP foram consolidadas no adendo `NACO_DECISOES_COMPLEMENTARES_MVP.md` versão 1.2 e propagadas nos documentos-base. Os itens desta seção são tarefas de materialização ou definições residuais; não reabrem escolhas já aprovadas.

18.1. Contratos que ainda precisam de detalhamento antes do freeze da OpenAPI

ID	Item	Situação / ação	Gate
C-01	Rotas e DTOs de categorias	Definir consulta/gestão com ownership por restaurante; domínio e vínculo já definidos.	API de catálogo
C-02	Upload de imagem do responsável	Definir path, request/response e limite/erros; Cloudinary/StoragePort já escolhidos.	API de mídia
C-03	Reprocessamento administrativo de exclusão Cloudinary	Definir procedimento e eventual contrato privado; Q77–Q86 já adaptadas.	Operação de mídia

ID	Item	Situação / ação	Gate
C-04	Dashboard multi-restaurante	`restaurantId` obrigatório em GET `/api/restaurant/dashboard`; detalhar filtros e DTO agregado.	API de dashboard
C-05	Visualização de pedidos	POST `/api/orders/{orderId}/views` já escolhido; detalhar schema de resposta e erros.	API de pedidos
C-06	Filtros admin de mídia	Registrar nomes, tipos e paginação da lista de operações.	API admin

18.2. Materialização de schema e implementação — trabalho futuro esperado

- Schema Prisma/PostgreSQL e migrations para as entidades lógicas aprovadas (`RoleRequest`, `RoleAuditEvent`, `OrderView`, `IdempotencyRecord`, operações de mídia e demais entidades de domínio).
- Constraints e índices físicos: ADMIN único/exclusivo; uma `RoleRequest` PENDING por usuário; unicidade de oferta por (`restaurant_id, normalized_name`); `OrderView` por (`order_id, responsible_user_id`); integrity entre oferta/categoria.
- Implementação de backend, mobile, web, Cloudinary, Redis, observabilidade e CI/CD conforme a stack decidida.
- Criar `docs/api/openapi.yaml` com base nos contratos já definidos. O arquivo ainda não integra o ZIP porque está previsto para a etapa de desenvolvimento; sua ausência não significa que os requisitos/rotas principais não tenham decisão.

18.3. Validações operacionais antes de staging/produção

- Implementar guards de navegação por papel nos clientes e seletor de restaurante no dashboard para responsáveis com múl-

tiplos estabelecimentos. A conta não troca de identidade/papel no login.
- Definir comportamento fail-safe/degraded quando Redis indisponível para cache e rate limiting, sem bypass silencioso de controles de segurança.
- Documentar evidência e procedimento externo de autorização do proprietário para ações ADMIN sujeitas a autorização prévia.
- Definir moderação/denúncia e retenção de avaliações públicas antes da abertura irrestrita de publicação; a consulta pública sem login e o rótulo “Cliente verificado” já estão aprovados.
- Cloudinary: validar a implementação de monitoramento de créditos com dados atualizados periodicamente; cachear consultas, respeitar limites da Admin API e bloquear novos uploads preventivamente. O patamar de 90% é uma salvaguarda técnica recomendada/configurável, não altera a decisão de bloquear no limite e não interrompe entrega das imagens existentes.
- Revalidar SLO/RPO/RTO, backups, retenção, métricas e alertas após o primeiro staging.
- Executar os 130 testes RNF-01, cenários de segurança, integração, concorrência, recuperação e homologação; registrar resultados reais.

A ausência de OpenAPI, schema físico, migrations, código e resultados de teste é compatível com o estágio de preparação do projeto. Não declara software implementado nem constitui inconsistência documental.

18.4. Decisões fechadas que não devem voltar à lista de pendências

- Uma conta por pessoa; papéis CLIENTE/RESPONSAVEL coexistem; ADMIN é único e exclusivo; sem seletor de identidade no login, pois mobile/web são superfícies por papel e o backend valida autorização.
- `RoleRequest`, `RoleAuditEvent`, no máximo uma solicitação PENDING por usuário e transferência de todos os restaurantes antes de remover RESPONSAVEL.
- Horários semanais com timezone IANA e intervalos `HH:mm`; feriados/exceções fora do MVP.
- BRL, `DECIMAL(12,2)` e strings decimais na API.
- `display_name`/`normalized_name` e unicidade normalizada por restaurante.
- Access JWT 15 min, refresh opaco rotativo até 30 dias, revogação e armazenamento seguro por plataforma.
- Carrinho local; métricas do dashboard e período já definidos.

- OpenAPI conventions: `camelCase`, `status`, envelope simbólico, códigos HTTP definidos no adendo.
- Avaliação PRODUCT_ONLY, leitura pública; rotas de review de produto aprovadas.
- GET pedidos são puros; POST view registra visualização; PATCH status altera estado.
- QU-60 resolvida em duas fronteiras transacionais; estoque só baixa na confirmação.
- Cloudinary Free via `StoragePort`, sem cartão/cobrança sem autorização; Q77–Q86 adaptadas para exclusão/invalidação best effort.

19. Estratégia de implementação por fatias verticais

Cada funcionalidade deve atravessar o sistema antes de passar à próxima:

1. Especificar contrato e casos de aceitação.
2. Atualizar OpenAPI e schemas de request/response.
3. Criar migration/constraints necessárias.
4. Implementar domínio e políticas.
5. Implementar repository/transação.
6. Implementar service/use case.
7. Implementar controller/route/middleware.
8. Testar unidade, integração e API.
9. Integrar frontend (componentes, estados e UX de falha).
10. Executar regressão e revisar segurança/observabilidade.

Não construir telas baseadas em DTOs provisórios. Em fluxo crítico, não considerar a tarefa concluída apenas porque a tela funciona com mocks.

20. Definição de pronto (DoD) e gates

Uma atividade só pode ser considerada concluída quando: - comportamento aprovado está implementado sem quebrar RF/RN/RNF; - contrato OpenAPI e tipos correspondem ao comportamento real; - autorização e validação são verificadas no servidor; - migrations/constraints estão versionadas e testadas; - testes apropriados passam em ambiente reproduzível; - erros seguem envelope e códigos aprovados; - logs não contêm segredos/PII proibida; - interface trata loading, vazio, erro, retry e conflito sem falso sucesso; - documentação de operação/decisão foi atualizada; - revisão de regressão

não encontrou falha crítica/alta nos fluxos envolvidos.

20.1 Marcos de liberação

- **M0 — Fundação:** repo, ambiente, CI, contratos e convenções funcionais.
- **M1 — Backend seguro:** banco, migrations, autenticação, sessão, papel e autorização.
- **M2 — Catálogo operacional:** restaurantes, horários, categorias, produtos, nomes, imagens e stock.
- **M3 — Pedido consistente:** carrinho local conectado, checkout idempotente, transições, eventos e stock atômico.
- **M4 — Produto completo:** reviews, dashboard, inbox, admin/role requests.
- **M5 — Experiências integradas:** mobile cliente e web gestão conectados à API real.
- **M6 — Homologação:** regressão, segurança, restauração, RNF-01, relatório e staging aprovado.

21. Glossário

Termo	Significado
`ProductBase`	Identidade lógica compartilhável de um produto.
`RestaurantProduct`	Oferta específica de um restaurante, com preço, estoque e status próprios.
Snapshot	Cópia imutável dos dados relevantes no instante do pedido.
`OrderItem`	Linha de item de um pedido e origem da avaliação de produto.
Idempotência	Repetir a mesma intenção não cria efeito duplicado.
Tombstone	Registro mínimo persistente que prova que uma chave já foi usada/expirou.
Cache frio/aquecido	Cache limpo ou previamente carregado antes da campanha.
P95/P99	Percentis de latência que representam 95%/99% das observações abaixo do valor.

Termo	Significado
VU	Usuário virtual simulado em teste de carga.
RPO	Ponto máximo de perda de dados aceitável em recuperação.
RTO	Tempo-alvo máximo para restabelecer serviço.
DoD	Definition of Done, critérios de conclusão de uma atividade.
API freeze	Marco em que contratos públicos ficam estáveis para implementação integrada.

22. Fontes e governança documental

Este documento consolida e aprofunda, sem substituir, as fontes oficiais do projeto:

1. `01_ESPECIFICACAO_CONSOLIDADA.md` — RF-01 a RF-09, RN-01 a RN-09, RNF-01/RNF-02, critérios, testes e rastreabilidade.
2. `02_MODELO_DADOS_E_API.md` — entidades, relacionamentos, constraints, rotas preservadas e pendências de DTO.
3. `03_REGISTRO_DECISOES_RNF01.md` — QU-01 a QU-73 e metodologia de desempenho.
4. `04_RELATORIO_COMPARATIVO_E_AUDITORIA.md` — auditoria de preservação, conflitos e limites da revisão.
5. `NACO_DECISOES_COMPLEMENTARES_MVP.md` — decisões técnicas de 14 etapas, rotas, modelo, stack, segurança, operação e testes (02/11/2026).

Governança: mudança em requisito RF/RN/RNF exige atualização do documento de requisitos e registro de decisão. Mudança de contrato atualiza OpenAPI, DTOs, testes e documentação. Mudança de stack/infra registra ADR com contexto, opções, decisão e consequências.

Declaração final de estado: o MVP está documentado em nível suficiente para iniciar a implementação por fatias verticais, após resolver os gates contratuais identificados. Este documento não afirma que código, migrations, integrações, testes, campanhas RNF-01, segurança operacional ou produção já existam ou tenham sido validados.

Apêndices de rastreabilidade integral

Os apêndices a seguir preservam o detalhamento operacional originalmente aprovado nos documentos-base. Em caso de conflito de formatação, os identificadores e os critérios devem ser comparados com os arquivos oficiais versionados no repositório.

Apêndice A — Critérios de aceitação, cenários e rastreabilidade preservados

Critérios 1. RF-01 — Cadastro e autenticação

- Dado um e-mail ainda não cadastrado e dados válidos, quando a conta for criada, então deve existir exatamente uma entidade `User` e um papel `CLIENTE` associado.
- Dado e-mail já cadastrado, quando houver nova tentativa de cadastro, então não deve ser criada outra conta.
- Dadas credenciais válidas, o login autentica e retorna a sessão conforme contrato vigente.
- Dadas credenciais inválidas, a autenticação é recusada (exemplo original HTTP 401).
- Não é possível obter papel `ADMIN` por cadastro público.
- Fluxo de solicitação de `RESPONSAVEL` respeita aprovação e cooldown de 7 dias após rejeição.

Critérios 2. RF-02 — Avaliação de produto

- Cliente só avalia `OrderItem` pertencente a pedido seu.
- Pedido deve estar em `CONCLUIDO`.
- Nota deve estar entre 1 e 5; comentário pode estar ausente.
- Uma segunda avaliação para o mesmo `order_item_id` deve ser rejeitada.
- Visitante sem autenticação consulta avaliações públicas paginadas por produto; autor aparece como “Cliente verificado”, sem dados pessoais.
- Avaliação não armazena cópia redundante de customer/order/restaurant; vínculo é obtido por relações.

Critérios 3. RF-03 — Pedido demonstrativo

- Cliente deve estar autenticado e ter `CLIENTE`.
- Restaurante deve estar publicado e aberto.
- Pedido contém itens de um único restaurante, com quantidade de 1 a 20 por produto e sem duas linhas para o mesmo `restaurant_product_id`.

- Backend revalida preço/disponibilidade e calcula total.
- Pedido inicia `CRIADO` e mantém snapshot de nome/preço.
- Transições inválidas são recusadas; cada alteração de status cria `OrderEvent`.
- Cancelamento respeita matriz de ator/estado; estoque é restaurado apenas se havia sido baixado na confirmação.
- Idempotência impede duplicação conforme seção 5.1.

Critérios 4. RF-04 — Painel

- Apenas responsável autorizado consulta dados do restaurante.
- Tentativa de consultar restaurante alheio é negada.
- Os valores devem corresponder aos dados-fonte; as fórmulas/indicadores específicos ainda dependem de decisão.

Critérios 5. RF-05/RF-09 — Catálogo

- Responsável só cria/edita/remove produto/oferta dentro de restaurante autorizado.
- Preço > 0; estoque >= 0.
- Produto sem estoque permanece visível e indisponível.
- Produto referenciado por pedido não pode ter histórico invalidado.
- Alterar nome/preço não modifica snapshots antigos.
- Oferta só usa categoria do mesmo restaurante.
- Duplicidade de nome é validada por `normalized_name` calculado no servidor e unicidade `(restaurant_id, normalized_name)`, inclusive para ofertas desativadas.

Critérios 6. RF-06/RF-07 — Papéis e restaurantes

- Conta é única e e-mail globalmente único.
- Conta pode ser CLIENTE e RESPONSAVEL; ADMIN é exclusivo.
- ADMIN é único, criado manualmente, sem cadastro público e com senha hash.
- Responsável pode gerenciar vários restaurantes; cada restaurante tem um único responsável.
- Novo restaurante começa `UNPUBLISHED`.
- `GET /api/me/restaurants` lista somente restaurantes do usuário autenticado.
- Despublicação é bloqueada quando há pedidos ativos; painel permanece acessível quando despublicado.

Critérios 7. RNF-01

- Cada uma das 30 execuções oficiais atende P95/P99 e erro por operação; cada combinação tem 5 execuções individuais aprovadas.
- Execução com qualquer operação acima do limiar ou taxa de erro acima de 1% reprova a combinação.
- Campanha de concorrência não apresenta estoque negativo, duplicidade de consumo/pedido ou reconciliação divergente.
- Campanha de overhead preserva pares equivalentes e evidências; overhead não reprova por percentual próprio.
- Os cenários de falha e recuperação comprovam rollback pré-commit e recuperação pós-commit via chave.

Critérios 8. RNF-02

- Diante de falha de comunicação, a interface exibe mensagem compreensível.
- A interface não mostra sucesso quando a operação não foi confirmada.
- Retry automático é limitado a falhas transitórias/operações seguras; criação de pedido repete a mesma intenção com a mesma `Idempotency-Key`, sem gerar nova chave.

8. Cenários de teste relacionados

Cenários 1. Conta, papéis e autorização

1. Cadastro válido cria User e papel CLIENTE.
2. E-mail duplicado não cria uma segunda conta.
3. CLIENTE e RESPONSAVEL podem coexistir na mesma conta.
4. Tentativa de associar ADMIN junto a CLIENTE/RESPONSAVEL é bloqueada.
5. ADMIN não pode ser criado via rota pública e deve existir uma única conta ADMIN.
6. Pedido de papel RESPONSAVEL segue aprovação administrativa.
7. Rejeição inicia cooldown de 7 dias.
8. Remoção de RESPONSAVEL exige ação administrativa e gera auditoria.
9. Testar concessão, remoção, aprovação e rejeição com registro auditável.
10. Acesso a dados pessoais de outra conta é negado.

Cenários 2. Restaurante, catálogo e painel

1. Responsável sem papel tenta cadastrar/gerir restaurante: negar.

2. Responsável cadastra dois restaurantes: ambos ficam associados ao mesmo usuário.
3. Restaurante novo inicia UNPUBLISHED.
4. Usuário não autorizado tenta listar/editar/gerir restaurante alheio: negar.
5. Despublicação com pedidos em estado ativo: HTTP 409 conforme contrato de gestão documentado.
6. Despublicação sem pedido ativo: atualiza publication_status sem excluir histórico.
7. Gerenciar restaurante despublicado continua permitido.
8. Restaurante publicado porém fechado permanece consultável, sem aceitar novo pedido.
9. Criar oferta com preço <= 0 ou estoque < 0: rejeitar.
10. Produto com estoque zero permanece visível, mas indisponível.
11. Categoria de outro restaurante não pode ser usada na oferta.
12. Atualizar oferta não altera snapshots de pedidos antigos.
13. Produto já usado em pedido não perde referências históricas.
14. Responsável consulta somente indicadores do seu escopo.

Cenários 3. Pedido e avaliação

1. Visitante tenta criar pedido: negar.
2. Usuário sem CLIENTE tenta criar pedido: negar.
3. Pedido com produtos de restaurantes diferentes: rejeitar.
4. Pedido com quantidade fora de 1..20: rejeitar.
5. Duplicar `restaurant_product_id` em linhas separadas do mesmo pedido: rejeitar.
6. Criar pedido válido gera estado CRIADO e OrderItems com snapshots.
7. Falha ao criar pedido não deixa cabeçalho sem itens nem registro parcial.
8. Confirmação baixa estoque atomicamente.
9. Recusa não baixa estoque.
10. Cancelar em cada combinação papel/estado valida matriz.
11. Cancelar após baixa restaura estoque uma única vez.
12. Mudança inválida de status não persiste.
13. Cada mudança válida produz OrderEvent com antes/depois, ator e horário.
14. Cliente avalia item próprio após CONCLUIDO.
15. Avaliar item de outro cliente ou antes de CONCLUIDO: rejeitar.
16. Avaliação fora de 1..5: rejeitar.
17. Segunda avaliação para mesmo item: rejeitar.

Cenários 4. Idempotência e falhas

1. Mesma key + mesmo payload em sequência: não duplica pedido e retorna o resultado original.
2. Mesma key + payload diferente: HTTP 409, sem mutação.
3. Mesma key em requisições simultâneas: apenas uma execução.
4. Segunda chamada aguarda até 5 segundos; se primeira terminar, devolve resultado; caso contrário 202 com referência.
5. Consultar `GET /api/orders/idempotency/{key}` com usuário titular: retorna status/resultado conforme contrato a detalhar.
6. Usuário diferente tenta consultar chave alheia: não vaza existência/conteúdo.
7. Chave expirada após 24h é recusada por causa do tombstone permanente.
8. Nova intenção gera nova key.
9. Payload equivalente com itens em ordem diferente gera o mesmo hash canônico.
10. Payload com campo relevante alterado gera hash diferente e 409.
11. Falha antes do commit gera rollback integral.
12. Resposta HTTP perdida após commit é recuperada pela idempotência.
13. Reiniciar processo com operação pendente e verificar estado persistido/recuperável.

Cenários 5. RNF-01: carga, concorrência e instrumentação

Executar os 30 testes oficiais, 60 execuções adicionais de overhead e 40 execuções de disputa de estoque conforme seção 5.1. Guardar logs/evidências individuais e reconciliação. Não substituir os limites aprovados por médias agregadas.

Cenários 6. RNF-02

1. API indisponível: mostrar mensagem compreensível, sem falsa confirmação.
2. Timeout: informar estado incerto sem disparar automaticamente uma nova intenção de compra com nova chave.
3. Erro de servidor: não apresentar sucesso.
4. Retry manual: respeitar futuras regras de repetição e idempotência quando forem especificadas.

9. Dependências entre requisitos

Origem	Depende de	Impacto/dependência
RF-01	`User`, `UserRole`	Conta única e autenticação são pré-condição para fluxos protegidos.
RF-02	RF-01, RF-03; `Order`, `OrderItem`, `Review`	Avaliação exige cliente autenticado, pedido próprio concluído e item comprado.
RF-03	RF-01, RF-07, RF-08; catálogo, horários, publicação, `Order`, `OrderItem`, `OrderEvent`	Pedido depende de cliente, restaurante válido, ofertas e disponibilidade.
RF-04	RF-07, RF-08; `Restaurant`, `Order`, `OrderItem`, `Review`	Painel depende de dados de restaurante, pedidos e avaliações; fórmulas e janela temporal definidas no adendo complementar.
RF-05	RF-06, RF-07; `ProductBase`, `RestaurantProduct`, `Category`	Gestão de produtos depende de papel RESPONSAVEL e ownership do restaurante.
RF-06	RF-01; `UserRole` e fluxo de solicitação/auditoria	Atribuição de RESPONSAVEL depende de aprovação administrativa; `RoleRequest` limita a uma solicitação PENDING por usuário e registra auditoria em `RoleAuditEvent`.
RF-07	RF-06; `User`, `UserRole`, `Restaurant`, `Order`	Gestão de restaurante depende de papel e associação do responsável.

Origem	Depende de	Impacto/dependência
RF-08	RF-03; `RestaurantProduct`, `Order`, `OrderItem`, `OrderEvent`	Estoque/preço participam da validação do pedido e das transições de confirmação/cancelamento.
RF-09	RF-05, RF-08; `ProductBase`, `RestaurantProduct`, `Category`, `OrderItem`	Alterações de catálogo não podem reescrever snapshots.
RNF-01	RF-01, RF-03, RF-04, RF-05, RF-07, RF-08; staging, banco, gerador, dados	Testa nove operações, concorrência, idempotência e integridade sob carga.
RNF-02	Todos os RF via API/cliente; Axios e contratos de erro	Falhas não podem ser mostradas como sucesso; retry e contratos precisam ser detalhados.

10. Matriz de rastreabilidade resumida

Requisito	Regras principais	Entidades	Endpoints documentados/relacionados	Testes principais
RF-01	RN-04 e regras de conta/papéis	`User`, `UserRole`	`POST /api/auth/register`; `POST /api/auth/login`; `POST /api/auth/logout`	Cadastro, e-mail único, login, logout, papéis e ciclo de sessão.

Requisito	Regras principais	Entidades	Endpoints documentados/relacionados	Testes principais
RF-02	RN-05	Review, OrderItem, Order, User	POST /api/order-items/{orderItemId}/reviews; GET /api/products/{productId}/reviews público	Elegibilidade, pedido entregue, ownership, nota, unicidade, leitura pública e privacidade do autor.
RF-03	RN-01, RN-06, RN-07, RN-08, RN-09; cancelamento/idempotência	Order, OrderItem, OrderEvent, Restaurant, RestaurantProduct	POST /api/orders/snapshot; GET pedidos/inbox/detalhes; POST /api/orders/{id}/cancel; PATCH /api/orders/{id}/status	Criação, snapshots, transições, cancelamento, eventos, idempotência e falhas.
RF-04	Permissões de responsável/admin	Restaurant, Order, OrderItem, Review, RestaurantProduct	GET /api/restaurants/dashboard?restaurantId={rest	Escopo autorizado, produtos vendidos e avaliações recebidas com fórmulas e períodos definidos.
RF-05	RN-02, RN-03, RN-06, RN-07	ProductBase, RestaurantProduct, Category, OrderItem	POST /api/restaurants/{id}/products; PATCH /api/products/{id}; DELETE /api/products/{id}	CRUD, ownership, duplicidade, soft delete, estoque zero e histórico.

Requisito	Regras principais	Entidades	Endpoints documentados/relacionados	Testes principais
RF-06	Conta SINGLE, CLIENTE automático, RESPONSAVEL request/approval, ADMIN exclusivo	`User`, `UserRole`, `RoleRequest`, `RoleAuditEvent`	`/api/me/role-requests`; `/api/admin/role-requests`; transferência e remoção admin	Cadastro de papel, aprovação, rejeição, cooldown, exclusividade e auditoria.
RF-07	Responsável único por restaurante, vários restaurantes por responsável; publicação	`User`, `UserRole`, `Restaurant`, `Order`	Endpoints de gerenciamento listados na seção API	Criação, listagem, gestão, publicação, despublicação, pedidos ativos.
RF-08	RN-06, RN-08; estoque na confirmação/estorno no cancelamento	`RestaurantProduct`, `Order`, `OrderItem`, `OrderEvent`	Rotas de produto; `POST /api/orders/{id}/cancel`; `PATCH /api/orders/{id}/status`	Estoque não negativo, disputa; baixa/restauração (total/cash)
RF-09	RN-02, RN-03, RN-07	`ProductBase`, `RestaurantProduct`, `Category`, `OrderItem`	`PATCH /api/products/{id}`; rotas de produto	Atualização, categoria no mesmo restaurante, snapshots.
RNF-01	QU-01 a QU-73	Todas as entidades envolvidas nos cenários	Nove operações da QU-01; endpoint idempotência aprovado	30 carga + 60 overhead + 40 concorrência, recuperação e reconciliação.

Requisito	Regras principais	Entidades	Endpoints documentados/relacionados	Testes principais
RNF-02	Tratamento de falhas	API, frontend, sessão	Endpoints via Axios e política de retry do adendo	Retry/backoff e regras de repetição segura definidos; validar implementação/testes.

11. Itens ainda a materializar (estado consolidado)

As decisões de domínio citadas nesta matriz foram fechadas no adendo complementar v1.2 e nos capítulos 5–10 deste documento. Esta lista não reabre essas decisões.

1. Criar `docs/api/openapi.yaml` a partir dos contratos orientadores; artefato planejado para a etapa de desenvolvimento.
2. Definir contratos de categoria, upload de imagem pelo responsável e operação administrativa de reprocessamento Cloudinary antes de implementar essas operações.
3. Materializar schema Prisma/PostgreSQL, constraints, índices e migrations das entidades lógicas já descritas.
4. Documentar e implementar moderação/denúncia e retenção de avaliações públicas, sem mudar sua leitura pública aprovada.
5. Definir comportamento fail-safe quando Redis estiver indisponível para rate limiting/cache.
6. Finalizar parâmetros operacionais (limites WebP, timeout/CPU/memória, consulta/cache da Usage API Cloudinary, alarmes e procedimento de autorização externa de ações ADMIN).
7. Executar a bateria de testes, campanhas RNF-01, testes de segurança, recuperação, carga e homologação. Nenhum resultado é antecipado neste pacote.
8. Ajustar SLO/RPO/RTO e política operacional após o primeiro staging, usando os baselines aprovados até lá.

A ausência da OpenAPI, schema físico, migrations, código e resultados executados é compatível com o estágio de preparação do projeto; não constitui pendência de decisão de produto nem evidência de implementação.

Apêndice B — Registro integral das decisões QU-01 a QU-73

QU-01 a QU-19 — Critérios centrais de desempenho

ID	Decisão aprovada
QU-01	Operações medidas: listar restaurantes; detalhes de restaurante; listar produtos; detalhes de produto; login/autenticação; criar pedido; pedidos do cliente; pedidos recebidos pelo restaurante; painel de gerenciamento. Não adicionar outra operação sem revisão. A rota de pedidos recebidos mantém conflito de ator/finalidade.
QU-02	P95: até 2 s para operações comuns; até 3 s para criação de pedido.
QU-03	Taxa máxima de erros técnicos de 1%; contar HTTP 5xx, timeouts e falhas inesperadas; excluir recusas legítimas como 409 de estoque insuficiente. 401 inesperado em fluxo autenticado deve ser investigado.
QU-04	Carga sustentada por 60 minutos.
QU-05	P99: até 4 s para operações comuns; até 6 s para criação de pedido.
QU-06	Ramp-up de 10 minutos até 1.000 usuários simultâneos; depois 60 minutos sustentados. Duração mínima oficial de 70 minutos, fora prepara-ção/aquecimento/encerramento.
QU-07	Distribuição de carga qualitativa realista, com mais consultas/listagens e menos pedidos.

ID	Decisão aprovada
QU-08	Distribuição de requisições: listar restaurantes 20%; detalhes restaurante 10%; listar produtos 20%; detalhes produto 10%; login 5%; criar pedido 5%; pedidos cliente 10%; pedidos recebidos restaurante 10%; painel 10%. Total 100%. Modelar sessão/VUs sem novo login a cada chamada.
QU-09	Ambiente de staging/homologação isolado e representativo de produção.
QU-10	Documentar recursos efetivos por execução: CPU, RAM, instâncias, banco/versão/recursos/conexões, OS, runtime/dependências, rede, gerador, data, versão da aplicação e configuração; verificar representatividade.
QU-11	Usar dados progressivos em volumes pequeno, médio e grande.
QU-12	Volumes: usuários 500/5.000/20.000; restaurantes 20/100/500; produtos/ofertas 500/5.000/25.000; pedidos 2.000/20.000/100.000; avaliações 500/5.000/25.000. São dados cadastrados, não simultaneidade.
QU-13	Estados de pedidos no dataset: CRIADO 10%; CONFIRMADO_RESTAURANTE 10%; EM_PREPARO 10%; SAIU_PARA_ENTREGA 10%; CONCLUIDO 45%; RECUSADO 5%; CANCELADO 10%. Gerar por transições válidas e temporalmente coerentes.

ID	Decisão aprovada
QU-14	Paginação: 20 padrão, 100 máximo; testar primeira e páginas profundas; acima do máximo rejeitar ou limitar consistentemente; ordenação estável com desempate único; paginação não substitui índices.
QU-15	Se houver cache, separar cache frio/aquecido, limpar previamente o frio, medir aquecimento e hit/miss. Não exige cache nem tecnologia específica.
QU-16	Intervalo aleatório de 1–5 s entre ações do mesmo VU; não determina RPS. Registrar RPS efetivo.
QU-17	Cinco execuções por combinação volume/cache: 3 × 2 × 5 = 30 execuções oficiais; preservar resultados individuais e analisar média, mediana, dispersão e percentis.
QU-18	Mesmos critérios de latência e erros em todos os cenários, inclusive cache frio e todos os volumes.
QU-19	Todas as cinco execuções de cada combinação devem cumprir individualmente. Percentis por operação dentro de cada execução; não mascarar operação lenta por média agregada. Uma execução reprovada reprova a combinação.

QU-20 a QU-28 — RPS e concorrência de estoque

ID	Decisão aprovada
QU-20	Medir/documentar RPS efetivo, total e por operação, incluindo variação, concluídas, falhas, timeouts e relação com latência/VUs. Não há RPS mínimo.
QU-21	Criar cenários específicos de disputa simultânea por estoque, além da carga geral.
QU-22	Níveis de 10, 25, 50 e 100 tentativas simultâneas pela mesma oferta.
QU-23	Estoque inicial igual a 50% da demanda, arredondado para baixo, uma unidade por tentativa: 5, 12, 25 e 50 unidades, respectivamente.
QU-24	Dez repetições por nível; total de 40 execuções. Restaurar estoque antes de cada execução.
QU-25	Usar barreira para liberação simultânea; registrar liberação e início efetivo. Sincronização do gerador não garante simultaneidade no banco.
QU-26	Latência de concorrência: P95 até 3 s e P99 até 6 s desde a liberação da barreira até resposta HTTP.
QU-27	Tentativas sem estoque recebem HTTP 409 e não consomem unidades. 409 de recusa legítima não é erro técnico.
QU-28	Após cada execução, reconciliar pedidos, respostas e estoque; sem duplicação, estoque negativo, consumo indevido ou alteração parcial.

QU-29 a QU-39 — Idempotência, atomicidade e falhas

ID	Decisão aprovada
QU-29	Reenvio da mesma solicitação não pode duplicar pedido nem consumo; testar reenvios sequenciais e simultâneos.
QU-30	Cliente gera `Idempotency-Key` para cada intenção lógica; nova intenção usa nova chave; chave associada ao usuário e ao resultado; payload diferente com mesma chave deve ser rejeitado (409 recomendado e depois aprovado na QU-56); persistência/atomicidade com pedido e estoque deve ser garantida, observada a ressalva QU-60.
QU-31	Dados completos idempotentes retidos por 24 horas. Após expiração, requisição antiga não pode gerar compra inesperada.
QU-32	Para mesma chave simultânea, segunda chamada aguarda a primeira. Se exceder limite, comunicar processamento sem segunda operação. Não manter transação/bloqueio do banco aberto enquanto aguarda.
QU-33	Espera de até 5 s; se primeira concluir, devolver o mesmo resultado; senão responder como processamento. A espera não cancela primeira operação; resultado continua recuperável.
QU-34	Injetar falhas em todos os pontos críticos antes/durante/depois da gravação do pedido e atualização de estoque.
QU-35	Rollback integral se falha antes do commit, sem alteração parcial; se commit ocorreu e resposta HTTP se perdeu, recuperação pela idempotência.

ID	Decisão aprovada
QU-36	Injeção controlada de falhas de banco, interrupção do processo e perda de resposta HTTP. Distinguir falha pré-commit de resposta perdida pós-commit.
QU-37	Após reinicialização, verificar consistência; preservar transações confirmadas, descartar incompletas e recuperar operações por chave.
QU-38	Evidência por execução: cenário/ponto de falha, horário, chave, respostas HTTP, logs correlacionados, estado de pedido/estoque antes/depois e reconciliação, sem dados sensíveis.
QU-39	Zero inconsistências: divergência, pedido duplicado, estoque negativo ou consumo indevido reprova a execução. Erros técnicos não reprovam automaticamente se integridade e demais critérios passam; taxa de erro segue QU-03.

QU-40 a QU-50 — Preparação, observabilidade e overhead

ID	Decisão aprovada
QU-40	Restaurar estado inicial controlado antes de cada execução; validar dados/isolamento e preservar volume, distribuição de status e cache definidos.
QU-41	Monitorar CPU, memória, rede, conexões/consultas DB, pool, disco e métricas da aplicação.

ID	Decisão aprovada
QU-42	Métricas de recurso são diagnósticas, sem limites arbitrários; reprovação segue critérios funcionais/desempenho já definidos.
QU-43	Validar gerador para 1.000 VUs sem gargalo; geração distribuída permitida se necessário; documentar topologia e validar perfil/sincronização/carga.
QU-44	Medir overhead da instrumentação, manter configuração consistente e documentar impacto; não subtrair overhead automaticamente dos resultados oficiais.
QU-45	Comparar instrumentação completa e mínima em condições equivalentes: mesma infraestrutura, app, dados, carga, cache, duração e repetições. Mínima preserva rastreabilidade básica.
QU-46	Cinco pares por combinação: 3 volumes × 2 cache × 5 pares = 30 pares/60 execuções adicionais.
QU-47	Alternar ordem entre pares: completa/mínima, depois mínima/completa, repetindo o padrão.
QU-48	Antes de cada execução, aguardar estabilização: sem requisições pendentes, filas residuais e com recuperação dos recursos. Separar de restauração de dados.
QU-49	Calcular percentual de overhead por par; manter resultados individuais, média, mediana e dispersão. Analisar cada métrica separadamente.

ID	Decisão aprovada
QU-50	Overhead como indicador diagnóstico, sem limite fixo ou reprovação automática; investigar e documentar impactos relevantes.

QU-51 a QU-56 — Fluxo de consulta e escopo da chave

ID	Decisão aprovada
QU-51	Se a segunda requisição com mesma chave não obtiver resultado dentro de 5 s, responder HTTP 202 Accepted, indicar processamento e fornecer referência de consulta. Primeira operação continua.
QU-52	Criar endpoint específico GET `/api/orders/idempotency/{key}` para consultar estado e resultado; exige autenticação e valida titularidade.
QU-53	Após expiração, manter registro mínimo da chave e rejeitar novas requisições com ela; nova intenção exige nova chave.
QU-54	Registro mínimo/tombstone permanece armazenado permanentemente enquanto o sistema existir.
QU-55	Unicidade por usuário: combinação `user_id + idempotency_key`.
QU-56	Mesma chave com payload diferente retorna HTTP 409 Conflict e não executa nova operação.

QU-57 a QU-62 — Payload e persistência idempotente

ID	Decisão aprovada
QU-57	Normalizar payload, ordenar itens deterministicamente e calcular hash dos campos relevantes.
QU-58	Considerar restaurante, produtos, quantidades e demais campos que efetivamente alterem o pedido; ignorar formatação irrelevante. Lista exata de campos permanece para contrato/schema.
QU-59	Persistir chave, usuário, hash do payload, estado do processamento e referência/resultado do pedido.
QU-60	Garantir atomicidade do registro idempotente, criação do pedido e operações relacionadas ao estoque. **Nota histórica:** a ressalva foi resolvida depois pelo adendo complementar v1.2 em duas fronteiras transacionais; estoque baixa apenas na confirmação.
QU-61	Diferenciar internamente chave inexistente, expirada, pertencente a outro usuário e operação em processamento, sem expor informações indevidas.
QU-62	Para mesma chave concorrente, garantir processamento único; demais requisições recuperam o mesmo resultado ou estado.

QU-63 a QU-67 — Concorrência entre ações de pedido

ID	Decisão aprovada
QU-63	Testar confirmações simultâneas de pedidos disputando a mesma oferta, respeitando baixa no momento da confirmação.
QU-64	Testar concorrência entre confirmação e cancelamento, evitando baixa/restauração duplicada.
QU-65	Testar cancelamentos simultâneos do mesmo pedido; estoque não pode ser restaurado duas vezes.
QU-66	Testar transições simultâneas de status; preservar máquina de estados e permissões.
QU-67	Após operações concorrentes, reconciliar pedidos, eventos, estoque e histórico.

QU-68 a QU-70 — Recuperação

ID	Decisão aprovada
QU-68	Injetar falhas antes, durante e após confirmação, baixa de estoque e gravação de eventos.
QU-69	Validar recuperação e consistência após reinicialização do processo.
QU-70	Recuperar pela chave a operação cujo commit ocorreu, mas cuja resposta HTTP foi perdida.

QU-71 a QU-73 — Relatório e encerramento

ID	Decisão aprovada
QU-71	Relatório consolidado com resultados, percentis, erros, RPS, recursos e overhead.

ID	Decisão aprovada
QU-72	Preservar rastreabilidade por execução: configuração, versão, dados, logs e evidências.
QU-73	Considerar RNF-01 validado somente após executar e avaliar todos os cenários e critérios aprovados.

Resumo quantitativo

Campanha	Quantidade
Carga oficial	30
Comparação de overhead	60 adicionais
Disputa concorrente de estoque	40
Total planejado	**130**

As quantidades são planejamento de testes, não evidência de execução.